

Inference Optimization for Deep Models and Large Language Models (LLMs)

Quantization, Pruning, Distillation, Low-Rank Methods, and Runtime Systems

Learning goals

By the end of this lecture, you should be able to:

- ▶ explain **transformer inference**, including **prefill**, **decode**, and the **key-value (KV) cache**;
- ▶ distinguish the main levers: **quantization**, **pruning/sparsity**, **distillation**, and **low-rank structure/adaptation**;
- ▶ reason about **deployment bottlenecks** rather than proxies such as parameter count or **floating-point operations (FLOPs)** alone;
- ▶ read an optimization claim and ask the right question: *what bottleneck improved, on what hardware and runtime stack?*;
- ▶ choose a disciplined first move for a real deployment problem.

Bottom line: Inference optimization is jointly a **model-design problem** and a **systems problem**.

What inference mostly computes

- ▶ A large fraction of neural-network inference is repeated **linear algebra**: matrix multiplies plus elementwise nonlinearities.
- ▶ A basic layer is

$$h = Wx + b, \quad u = \sigma(h).$$

- ▶ Here x is the input vector, W the weight matrix, b the bias, and σ an elementwise nonlinearity such as the Gaussian Error Linear Unit (GELU) or ReLU.
- ▶ Most model-side efficiency methods change one of three things:
 - the entries of W ,
 - the precision used to store them,
 - or the way the resulting operations run on hardware.

Training vs. inference

- ▶ **Training** updates parameters using gradients and backpropagation.
- ▶ **Inference** keeps parameters fixed and evaluates the trained network on new inputs.

Because deployed services execute inference many times, even a small per-request saving matters operationally.

Bottom line: To optimize inference, track **which kernels repeat**, **where bytes move**, and **how a small gain compounds at scale**.

What a transformer block contains

A minimal transformer stack repeatedly combines four pieces:

- ▶ **Embedding:** maps token IDs to vectors.
- ▶ **Self-attention:** lets each token read information from other tokens.
- ▶ **Feed-forward network (FFN):** a positionwise nonlinear transformation.
- ▶ **Residual + normalization:** stabilize deep networks and preserve a clean signal path.

For one attention head,

$$Q = HW_Q, \quad K = HW_K, \quad V = HW_V, \quad \text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

- ▶ H is the matrix of token hidden states; W_Q, W_K, W_V are learned projections.
- ▶ d_k is the key dimension; dividing by $\sqrt{d_k}$ keeps scores numerically well scaled.
- ▶ **Interpretation:** each token forms query/key/value vectors, scores other tokens, then reads a weighted sum of their values.
- ▶ **Why it matters for inference:** embeddings and FFNs are large dense matrix multiplies, while attention adds heavy tensor movement and growing KV-cache traffic.

Bottom line: A transformer is a repeated stack of dense linear algebra plus attention-specific memory movement.

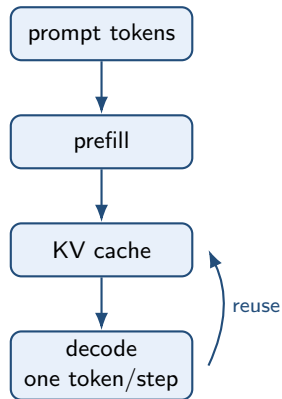
Autoregressive generation: prefill, decode, and KV cache

- ▶ **Prefill:** process the prompt and build the **KV cache** of past **keys** and **values**.
- ▶ **Decode:** generate one new token at a time while reusing the KV cache.
- ▶ Prefill often benefits from batching and high arithmetic utilization.
- ▶ Decode often becomes **memory-bandwidth bound**: each step rereads weights and cached state.

$$T_{\text{req}} = T_{\text{prefill}} + \sum_{t=1}^{L_{\text{out}}} T_{\text{decode},t} + \text{other overheads.}$$

L_{out} is the number of generated tokens. Long prompts stress prefill; long outputs stress decode.

Bottom line: Report **prefill** and **decode** separately; one “latency” number hides the bottleneck.



What deployment actually measures

Quantity	Meaning	Why it matters
Model memory	Bytes to store weights and metadata	Determines whether the model fits and how many copies can be served
KV-cache memory	Bytes for cached keys/values during generation	Dominates long-context serving and high concurrency
Latency	Wall-clock time for one request	What a single user waits for
Throughput	Requests/s or tokens/s at a stated load	What the service can finish per second
Energy	Joules/request or tokens/joule	Operating cost and thermal budget

- ▶ For interactive LLMs, also separate **TTFT** (time to first token) and **TPOT/ITL** (time per output token / inter-token latency).
- ▶ A benchmark must state the **operating point**: prompt length, output length, batch size or concurrency, hardware, and software stack.

Bottom line: “Efficient” is not one number. Memory, latency, throughput, and energy can move in different directions.

Why fewer FLOPs may not reduce latency

A useful lower bound for a kernel or layer is

$$T_{\text{layer}} \gtrsim \max\left(\frac{F}{P_{\text{eff}}}, \frac{B}{BW_{\text{eff}}}\right) + T_{\text{overhead}}.$$

- ▶ F : arithmetic operations.
- ▶ B : bytes moved through the memory hierarchy.
- ▶ P_{eff} : effective compute rate; BW_{eff} : effective bandwidth.
- ▶ **Interpretation:** runtime is limited by whichever term is larger: **compute** or **memory traffic**.
- ▶ **Why it matters:**
 - quantization often helps because it reduces **bytes moved**;
 - unstructured sparsity often disappoints because it reduces nominal FLOPs but may not reduce the actual bandwidth bottleneck.

Deployment rule

Lower parameter count or lower theoretical FLOPs alone do *not* guarantee lower wall-clock latency.

One running example for all four levers

Start with a tiny linear layer:

$$W = \begin{bmatrix} 0.9 & -0.2 \\ 0.1 & 1.3 \end{bmatrix}, \quad x = \begin{bmatrix} 2 \\ -1 \end{bmatrix}, \quad y = Wx = \begin{bmatrix} 2.0 \\ -1.1 \end{bmatrix}.$$

- **Quantization:** coarse rounding can change the output from $[2.0, -1.1]^\top$ to about $[2.0, -1.5]^\top$.
- **Pruning:** zeroing the smallest weight 0.1 changes the output to about $[2.0, -1.3]^\top$.
- **Low rank:** keep only the strongest interaction directions.
- **Distillation:** teach a smaller model to match a teacher distribution such as $[0.70, 0.25, 0.05]$.

Why keep this example in mind?

Every method either changes the model **representation/structure** while trying to preserve outputs, or trains a **new smaller model** to imitate the original one.

The four levers at a glance

Family	What changes?	Why can it help?	Main catch
Quantization	Numeric precision of weights, activations, or KV cache	Fewer bytes; sometimes faster kernels	Accuracy loss; kernel support matters
Pruning / sparsity	Which weights, channels, heads, or blocks remain	Smaller model or less useful work	Many sparse patterns do not speed up dense hardware
Distillation	Training targets and usually the deployed architecture	Dense small students can deploy very well	Requires retraining and a teacher
Low-rank structure	Matrix factorization or low-rank updates	Fewer degrees of freedom; cheap adaptation	Too-small rank can underfit or help training more than inference

Bottom line: The families differ mainly in **what they change**, **when they require training**, and **whether the result maps cleanly to fast kernels**.

Deployment-constrained optimization

A clean abstraction is to view deployment as constrained optimization over a transformed model $\tilde{\theta}$ and runtime policy π :

$$\begin{aligned} \min_{\tilde{\theta}, \pi} \quad & \mathcal{L}_{\text{task}}(\tilde{\theta}) \\ \text{s.t.} \quad & M(\tilde{\theta}) \leq B_M, \\ & T_{p95}(\tilde{\theta}, \pi, \mathcal{H}) \leq B_T, \\ & Q(\tilde{\theta}, \pi, \mathcal{H}) \geq B_Q. \end{aligned}$$

- ▶ M : memory footprint; T_{p95} : p95 latency; Q : throughput.
- ▶ $\mathcal{H}$ is the target hardware and runtime stack.
- ▶ **Interpretation:** a method is useful only if the final deployed configuration satisfies real memory, latency, and throughput budgets.
- ▶ Quantization changes the feasible numeric representation.
- ▶ Pruning changes the allowed topology or sparsity pattern.
- ▶ Low-rank methods constrain the matrix or update rank.
- ▶ Distillation changes the training objective so that a smaller model can solve the task.

Why second-order structure keeps reappearing

Near a trained model θ , a small perturbation δ gives

$$\mathcal{L}(\theta + \delta) \approx \mathcal{L}(\theta) + g^\top \delta + \frac{1}{2} \delta^\top H \delta.$$

At a local optimum, $g \approx 0$, so the leading loss change is

$$\Delta \mathcal{L} \approx \frac{1}{2} \delta^\top H \delta.$$

- ▶ H is the Hessian: a local curvature or sensitivity matrix.
- ▶ **Interpretation:** not all perturbations are equally harmful; directions with larger curvature are more expensive to change.
- ▶ This is why Hessian surrogates or activation covariances keep appearing in:
 - GPTQ (Accurate Post-Training Quantization for Generative Pre-trained Transformers)-like post-training quantization,
 - Optimal Brain Damage (OBD) / Optimal Brain Surgeon (OBS)-style pruning,
 - mixed-precision allocation,
 - low-rank approximation criteria.

Bottom line: The methods differ mostly in the **constraint set**; the local loss penalty often has the same quadratic form.

The bottleneck principle and a decision rule

Bottleneck principle

An optimization is worth deploying only if it improves the **actual bottleneck term** on the target stack.

- ▶ If decode is bandwidth-bound, reducing weight bytes or KV-cache traffic can matter more than lowering arithmetic count.
- ▶ If the runtime still dispatches dense kernels, generic unstructured sparsity may reduce model size but not latency.
- ▶ If the architecture may change, a dense distilled student may beat a heavily compressed large model in end-to-end serving.

Three deployment tests

A candidate optimization should:

- 1 preserve task quality on the relevant evaluation distribution;
- 2 improve the bottleneck term for the target workload;
- 3 be measured under the **same** stack, workload, and quality target as the baseline.

Quantization: what changes, and why it often helps first

Quantization stores tensors in fewer bits: for example 16-bit floating point (FP16) $\rightarrow$ 8-bit integer (INT8) or 4-bit integer (INT4).

- ▶ Targets can include **weights**, **activations**, and the **KV cache**; each target attacks a somewhat different bottleneck.
- ▶ Raw storage for a tensor with N entries at b bits is about $Nb/8$ bytes.
- ▶ Moving from FP16 to INT4 divides raw storage by roughly 4.

For autoregressive models, KV-cache memory scales approximately as

$$M_{KV} \approx 2B n_{\text{layers}} n_{\text{heads}} d_{\text{head}} T \cdot \frac{b_{KV}}{8}.$$

- ▶ B : active sequences; n_{layers} : transformer layers; n_{heads} : attention heads.
- ▶ d_{head} : width of each head; T : cached tokens; b_{KV} : bits per stored key/value.
- ▶ **Interpretation**: long-context serving can become KV-cache dominated, so KV-cache quantization is a first-class systems lever.
- ▶ Weight-only quantization often already helps decode because weights are reread every step; activation or KV-cache quantization matters when those tensors dominate traffic.

Bottom line: Quantization often helps first because it attacks the **bytes-moved term** directly, especially in bandwidth-bound decode.

Affine quantization by hand

For one scalar x , affine quantization uses a grid

$$\hat{x} = s(q - z), \quad q \in \{q_{\min}, \dots, q_{\max}\}.$$

A nearest-grid quantizer is

$$q^* = \text{clip}\left(\left\lfloor \frac{x}{s} \right\rfloor + z, q_{\min}, q_{\max}\right), \quad \hat{x} = s(q^* - z).$$

- ▶ $s > 0$ is the **scale**; z is the **zero-point**; q is the stored integer code.
- ▶ **Symmetric quantization**: $z = 0$, so $\hat{x} = sq$.
- ▶ **Asymmetric quantization**: nonzero z lets real zero be represented exactly even when the range is not symmetric.

Scalar example

If $x = 0.93$, $s = 0.25$, and the nearest code is $q = 4$, then $\hat{x} = 1.00$. If $x = 2.3$ is clipped to the largest representable code, the error can become much larger.

Bottom line: Quantization error comes from both **rounding** and **clipping**; outliers make clipping especially costly.

Worked scalar quantization: rounding versus clipping

Assume **symmetric** quantization with

$$z = 0, \quad s = 0.25, \quad q \in \{-4, -3, \dots, 4\}.$$

So the largest representable magnitude is $sq_{\max} = 0.25 \cdot 4 = 1.00$.

Example A: rounding inside range

$$\frac{x}{s} = \frac{0.93}{0.25} = 3.72$$

$$q^* = \text{round}(3.72) = 4$$

$$\hat{x} = 0.25 \cdot 4 = 1.00$$

$$\text{error} = \hat{x} - x = 1.00 - 0.93 = 0.07$$

Inside range, so only rounding matters.

Example B: clipping outside range

$$\frac{x}{s} = \frac{2.30}{0.25} = 9.2$$

$$\text{round}(9.2) = 9, \quad \text{but } q_{\max} = 4$$

$$q^* = \text{clip}(9, -4, 4) = 4$$

$$\hat{x} = 0.25 \cdot 4 = 1.00$$

$$\text{error} = 1.00 - 2.30 = -1.30$$

Outside range, so clipping dominates.

Bottom line: Rounding error is usually local and small; clipping error can explode once a value falls outside the chosen quantization range.

Scale, zero-point, and calibration

To map a real interval $[a, b]$ to integer codes $[q_{\min}, q_{\max}]$,

$$s = \frac{b - a}{q_{\max} - q_{\min}}, \quad z \approx q_{\min} - \frac{a}{s}.$$

If clipping is small and values are roughly uniform inside each quantization cell, then the error $e = \hat{x} - x$ obeys

$$\mathbb{E}[e] \approx 0, \quad \mathbb{E}[e^2] \approx \frac{s^2}{12}.$$

- ▶ **Interpretation:** smaller scales reduce rounding noise, but overly small scales increase clipping.
- ▶ **Calibration** is the problem of choosing ranges that balance those two errors.

Practical choices

- ▶ per-tensor scales: cheapest metadata, weakest fit to heterogeneous channels;
- ▶ per-channel or grouped scales: better accuracy, more metadata and more complex kernels;
- ▶ asymmetric activations are common; symmetric weights are often more hardware friendly.

Why integer-only inference was a breakthrough

If activations and weights are approximated by

$$x_i \approx s_x(q_{x,i} - z_x), \quad w_i \approx s_w(q_{w,i} - z_w),$$

then a dot product becomes

$$\sum_i w_i x_i \approx s_w s_x \sum_i (q_{w,i} - z_w)(q_{x,i} - z_x).$$

Expanding the product gives integer accumulations plus small correction terms:

$$\sum_i q_{w,i} q_{x,i} - z_w \sum_i q_{x,i} - z_x \sum_i q_{w,i} + dz_w z_x.$$

- ▶ d is the vector length.
- ▶ **Interpretation:** most of the work can happen in integer arithmetic, followed by a small requantization step.
- ▶ **Why it matters:** quantization became a genuine deployment primitive, not merely a storage trick, once kernels could execute this path efficiently.

Bottom line: Quantization pays off when the runtime actually uses low-bit kernels instead of silently dequantizing back to high precision.

Post-training quantization (PTQ) vs. quantization-aware training (QAT)

Question	PTQ	QAT
When applied?	After full-precision training	During fine-tuning or retraining
Signal used	Calibration stats, reconstruction, Hessian surrogates	End-to-end task loss with fake quantization in the loop
Cost	Low	Higher, often much higher
Strength	Fast conversion, good for INT8 or weight-only INT4	Better recovery for aggressive low-bit settings
Failure mode	Bad calibration, outliers, kernel mismatch	Optimization instability, overfitting, training cost
When preferred?	Need quick deployment or no retraining budget	PTQ loses too much accuracy and training budget exists

- ▶ **PTQ**: project a trained model onto a low-bit representation.
- ▶ **QAT**: train the model to live with the quantized forward pass.

From scalar quantization to layer error

For a linear layer $y = Wx$, the quantized forward pass is $\hat{y} = \hat{W}\hat{x}$. The output error decomposes as

$$\hat{y} - y = (\hat{W} - W)x + \hat{W}(\hat{x} - x).$$

- ▶ The first term is **weight quantization error**.
- ▶ The second term is **activation quantization error** after it passes through the quantized weights.
- ▶ **Interpretation:** weight-only quantization is often easier because weights are fixed, while activations vary with the input distribution and can contain sharp outliers.
- ▶ This is also why **KV-cache quantization** is its own design choice: the cache behaves like an input-dependent activation store, not like a fixed weight matrix.

Why this matters

The decomposition explains why many successful LLM deployments start with **weight-only** PTQ and add activation or KV-cache quantization only when the workload really needs it.

PTQ as layer-output reconstruction

For a linear layer with weights $W \in \mathbb{R}^{m \times d}$ and calibration activations $X \in \mathbb{R}^{d \times n}$, a canonical PTQ objective is

$$\hat{W} = \arg \min_{\hat{W} \in \mathcal{Q}} \|WX - \hat{W}X\|_F^2.$$

- ▶ X collects representative inputs from a calibration set.
- ▶ $\mathcal{Q}$ is the feasible low-bit codebook or quantized parameter set.
- ▶ **Interpretation:** PTQ should preserve the **layer output on representative inputs**, not merely make weights numerically close coordinatewise.

For one row w , writing $\delta = \hat{w} - w$ gives

$$\|wX - \hat{w}X\|_2^2 = \delta(XX^\top)\delta^\top.$$

- ▶ The activation covariance $H = XX^\top$ acts like a local Hessian surrogate.
- ▶ This is the bridge from simple PTQ to GPTQ-like second-order methods.

GPTQ-style compensation: why second-order PTQ helps

Suppose one weight coordinate w_j is fixed to a quantized value q_j , and the remaining coordinates are allowed to compensate. The local problem yields the update

$$w \leftarrow w - \frac{w_j - q_j}{[H^{-1}]_{jj}} H_{:,j}^{-1}.$$

The minimum added reconstruction error is

$$\Delta \approx \frac{(q_j - w_j)^2}{[H^{-1}]_{jj}}.$$

- ▶ H is the local activation-covariance or Hessian surrogate.
- ▶ $H_{:,j}^{-1}$ is column j of the inverse surrogate.
- ▶ **Interpretation:** after quantizing one coordinate, update the unfixed coordinates to cancel part of the induced output error.

Why this matters for practice

This is the core idea behind **GPTQ**: quantize aggressively, but compensate in the directions the layer can most cheaply absorb.

Deriving the GPTQ compensation update

Why this objective? For one row, the previous slide gives the local reconstruction loss $\delta^\top H \delta$ with $\delta = \hat{w} - w$. GPTQ quantizes one coordinate at a time, so once coordinate j is forced to q_j , the remaining free coordinates should minimize that same local loss:

$$\min_{\delta} \delta^\top H \delta \quad \text{s.t.} \quad e_j^\top \delta = q_j - w_j, \quad \delta = \hat{w} - w.$$

$$\mathcal{J}(\delta, \lambda) = \delta^\top H \delta + \lambda(e_j^\top \delta - (q_j - w_j)),$$

$$\nabla_{\delta} \mathcal{J} = 0 \Rightarrow 2H\delta + \lambda e_j = 0 \Rightarrow \delta = -\frac{\lambda}{2} H^{-1} e_j = -\frac{\lambda}{2} H_{:,j}^{-1}.$$

Now apply the constraint:

$$q_j - w_j = e_j^\top \delta = -\frac{\lambda}{2} [H^{-1}]_{jj} \Rightarrow \lambda = -\frac{2(q_j - w_j)}{[H^{-1}]_{jj}}.$$

Substitute back:

$$\delta^* = \frac{q_j - w_j}{[H^{-1}]_{jj}} H_{:,j}^{-1}, \quad \hat{w} = w + \delta^* = w - \frac{w_j - q_j}{[H^{-1}]_{jj}} H_{:,j}^{-1}.$$

Interpretation

Column j of H^{-1} is the cheapest local compensation direction after forcing coordinate j to its quantized value.

QAT: fake quantization and the straight-through estimator (STE)

QAT optimizes the task loss with a fake-quantized forward pass:

$$\min_{\theta, s} \mathbb{E}_{(x, y) \sim \mathcal{D}} [\ell(f_{\hat{\theta}(s)}(x), y)], \quad \hat{v} = s \operatorname{clip}\left(\left\lfloor \frac{v}{s} \right\rfloor, q_n, q_p\right).$$

- ▶ v is a scalar weight or activation; s is a fixed or learnable scale.
- ▶ q_n, q_p are the smallest and largest integer codes allowed by the chosen bitwidth.
- ▶ **Interpretation:** the forward pass sees a quantized value even though optimization keeps a higher-precision parameter copy.

Inside the unclipped region, STE uses the surrogate gradients

$$\frac{\partial \hat{v}}{\partial v} \approx 1, \quad \frac{\partial \hat{v}}{\partial s} \approx \left\lfloor \frac{v}{s} \right\rfloor - \frac{v}{s}.$$

- ▶ Outside the clipping range, the gradient with respect to v is usually treated as 0 or otherwise damped.
- ▶ **Historical note:** Parameterized Clipping Activation (PACT) makes the clipping limits trainable, while Learned Step Size Quantization (LSQ)-style QAT learns the scale s in $\hat{v} = s \operatorname{clip}(\lfloor v/s \rfloor, q_n, q_p)$.

Bottom line: PTQ projects a trained model into low-bit form; QAT trains the model to **live inside** the quantized forward path.

Practical quantization pitfalls and evaluation

Pitfall	What goes wrong	How to read the result
Outliers / clipping	A few extreme channels consume most of the representable range	Good clipping or a tiny mixed-precision escape hatch can matter more than one extra bit
Calibration mismatch	Short, clean, or too-small calibration sets miss real activation ranges	PTQ may look fine offline but fail on real deployment prompts
Granularity choice	Per-tensor scales are too coarse for heterogeneous channels	Per-channel or grouped scales often recover large accuracy gaps
Accumulator / requantization path	Overflow or poor rescaling corrupts low-bit arithmetic	Integer-only support is an end-to-end kernel issue, not just a storage issue
Kernel fallback	INT4/INT8 tensors are silently dequantized for compute	Memory can drop while latency barely moves
Weak evaluation	Only perplexity or only average latency is reported	Check downstream quality and same-stack end-to-end timing

- A credible quantization report should state **bitwidth, scale granularity, calibration protocol, and the actual kernel path**.

Bottom line: Most failed quantization deployments are caused by calibration and kernel issues, not by the affine formula itself.

Recent LLM quantization families

Family	Core move	Representative methods
Second-order PTQ	Quantize rows while locally compensating output error	GPTQ
Outlier-aware / rebalanced PTQ	Protect fragile channels or shift difficulty between activations and weights	LLM.int8(), SmoothQuant, Activation-aware Weight Quantization (AWQ)
Differentiable PTQ	Optimize clipping or equivalent transforms on calibration data	OmniQuant
Low-bit QAT recovery	Train under fake quantization when PTQ becomes too lossy	large language model quantization-aware training (LLM-QAT), BitDistiller
Rotations / preconditioning	Rotate coordinates so outliers are spread more evenly	QuaRot, SpinQuant

- **Organizing principle:** each family either reconstructs layer outputs better, handles outliers better, or changes coordinates to make uniform low-bit quantization easier.
- The paper names are **representatives**; the mechanism matters more than the chronology.

Bottom line: Modern LLM quantization is a small set of recurring ideas, not a long list of unrelated papers.

Three sparsity patterns to keep separate

Pattern	What is removed?	Deployment reading
Unstructured sparsity	Individual weights anywhere in the tensor	Most flexible for compression; weakest path to speedup on dense hardware
Structured sparsity	Whole rows, columns, channels, heads, or blocks	Produces smaller dense tensors; strongest path to direct latency gains
Semi-structured $N:M$	A fixed local rule such as 2:4	In every local block of M weights, exactly N remain nonzero

- ▶ **Example:** 2:4 means that in every group of 4 nearby weights, only 2 may stay nonzero.
- ▶ Unstructured sparsity gives the best flexibility, but it creates irregular memory access.
- ▶ Semi-structured sparsity sits in between: less flexible than arbitrary zeros, but regular enough for some sparse tensor-core kernels.

Bottom line: The practical question is not “how many zeros?” but **what pattern the zeros follow** and whether the runtime can exploit that pattern.

Pruning as constrained optimization

A canonical pruning objective is

$$\min_{m, W^c} \mathcal{L}(m \odot W^c) \quad \text{s.t.} \quad \|m\|_0 \leq k.$$

- ▶ $m \in \{0, 1\}^{\dim(W)}$ is a binary mask selecting which weights survive.
- ▶ W^c is the compensated / recovered weight tensor: after pruning, the surviving weights may shift from W to reduce damage.
- ▶ $m \odot W^c$ is the final sparse model: $m_i = 0$ forces coordinate i to zero, while $m_i = 1$ keeps the value from W^c .
- ▶ If no recovery is allowed, one can set $W^c = W$; with brief retraining or local compensation, the kept entries in W^c are re-optimized.
- ▶ $\|m\|_0$ is the “0-norm”: it counts nonzero mask entries, so $\|m\|_0 \leq k$ means at most k coordinates survive.

Why this framing is useful

Pruning has two design axes: the **pattern** (element, channel, head, block, ...) and the **recovery mechanism** (none, local compensation, brief fine-tuning, full retraining).

Optimal Brain Damage and magnitude pruning

If weight coordinate θ_i is removed with no compensation, then near a trained optimum

$$\Delta\mathcal{L}_i \approx \frac{1}{2} H_{ii} \theta_i^2.$$

- ▶ H_{ii} is the diagonal curvature for coordinate i . No full diagonal assumption is needed: pruning only coordinate i gives $\delta = -\theta_i e_i$ and $\delta^\top H \delta = H_{ii} \theta_i^2$.
- ▶ **Interpretation:** prune coordinates that are both small and locally insensitive.

If the diagonal curvatures are treated as roughly comparable, the ranking reduces to magnitude pruning:

$$\Delta\mathcal{L}_i \propto \theta_i^2.$$

- ▶ This explains why simple magnitude pruning is often a strong baseline.
- ▶ The stronger approximation enters here: magnitude-only ranking treats the diagonal curvatures as roughly comparable and ignores cross-coordinate interactions.

Bottom line: Magnitude pruning is the low-cost shadow of a second-order pruning argument.

Optimal Brain Surgeon: compensation matters

If coordinate i is forced to zero but other coordinates may compensate, the local solution is

$$\delta^* = -\frac{\theta_i}{[H^{-1}]_{ii}} H_{:,i}^{-1}, \quad \Delta \mathcal{L}_i^* = \frac{1}{2} \frac{\theta_i^2}{[H^{-1}]_{ii}}.$$

- ▶ $H_{:,i}^{-1}$ is column i of the inverse Hessian surrogate.
- ▶ **Interpretation:** if one weight is removed, adjust the remaining weights in the cheapest compensating direction.
- ▶ This is the pruning analogue of GPTQ-style compensation in PTQ.

Modern connection

SparseGPT applies OBS-style local reconstruction layerwise at LLM scale using activation-covariance surrogates and blockwise numerics.

Deriving the OBS compensation update

Objective setup. Near a trained optimum, a perturbation δ changes the loss by

$$\Delta\mathcal{L} \approx \frac{1}{2}\delta^\top H\delta.$$

Pruning coordinate i means the final value should be 0, so $\delta_i = -\theta_i$ while the other coordinates are free to compensate.

Objective.

$$\min_{\delta} \frac{1}{2}\delta^\top H\delta \quad \text{s.t.} \quad \mathbf{e}_i^\top \delta = -\theta_i.$$

Derivation.

$$\mathcal{J}(\delta, \lambda) = \frac{1}{2}\delta^\top H\delta + \lambda(\mathbf{e}_i^\top \delta + \theta_i), \quad \nabla_{\delta}\mathcal{J} = H\delta + \lambda\mathbf{e}_i = 0$$

$$\Rightarrow \delta = -\lambda H^{-1}\mathbf{e}_i = -\lambda H_{:,i}^{-1}, \quad -\theta_i = \mathbf{e}_i^\top \delta = -\lambda[H^{-1}]_{ii}$$

$$\Rightarrow \lambda = \frac{\theta_i}{[H^{-1}]_{ii}}, \quad \delta^* = -\frac{\theta_i}{[H^{-1}]_{ii}} H_{:,i}^{-1}$$

$$\Delta\mathcal{L}_i^* \approx \frac{1}{2}\delta^{*\top} H\delta^* = \frac{1}{2} \frac{\theta_i^2}{[H^{-1}]_{ii}}.$$

Bottom line: OBS prunes one coordinate, then moves in the cheapest compensating direction allowed by the local quadratic model.

Structured pruning: group sparsity gives a hardware path

For groups $G_1, \dots, G_G$ (channels, heads, blocks, ...), a standard formulation is

$$\min_W \mathcal{L}(W) + \lambda \sum_{g=1}^G \|W_{G_g}\|_2.$$

The proximal update for one group is

$$W_{G_g}^+ = \left(1 - \frac{\eta\lambda}{\|V_g\|_2}\right)_+ V_g, \quad V_g = W_{G_g} - \eta \nabla_{G_g} \mathcal{L}(W).$$

- ▶ **Interpretation:** groups with small post-gradient norm are shrunk all the way to zero.
- ▶ This naturally yields channel-, head-, or block-level pruning.
- ▶ **Why it matters for deployment:** removing groups changes tensor shapes, so the resulting model is still dense but smaller.
- ▶ **Network Slimming** is a classic example: train channel scale parameters so unimportant channels reveal themselves by near-zero gates.

Deriving the group sparsity proximal update

Objective setup. Proximal gradient first takes the smooth-loss step

$$V_g = W_{G_g} - \eta \nabla_{G_g} \mathcal{L}(W),$$

then solves, for one group,

$$\min_z \frac{1}{2\eta} \|z - V_g\|_2^2 + \lambda \|z\|_2.$$

Derivation. Because the penalty depends only on the group norm, the minimizer keeps the direction of V_g . Write

$$V_g = ru, \quad r = \|V_g\|_2, \quad \|u\|_2 = 1, \quad z = tu, \quad t \geq 0.$$

$$\min_{t \geq 0} \frac{1}{2\eta} (t - r)^2 + \lambda t \quad \Rightarrow \quad t^* = (r - \eta\lambda)_+.$$

Substitute back:

$$z^* = \frac{(r - \eta\lambda)_+}{r} V_g = \left(1 - \frac{\eta\lambda}{\|V_g\|_2}\right)_+ V_g.$$

Bottom line: Group lasso soft-thresholds the *whole group norm*, so entire channels, heads, or blocks disappear together.

Why unstructured sparsity often fails to speed up GPUs

For a dense matrix multiply with shapes $(m \times k)$ and $(k \times n)$,

$$F_{\text{dense}} = 2mkn.$$

If density is ρ , the *theoretical* sparse arithmetic count is

$$F_{\text{sparse}} \approx 2\rho mkn.$$

But end-to-end time is closer to

$$T_{\text{sparse}} \gtrsim \max\left(\frac{2\rho mkn}{P_{\text{sparse}}}, \frac{B_x + \rho B_w + B_{\text{idx}} + B_y}{BW_{\text{sparse}}}\right).$$

- ▶ ρ : nonzero density; P_{sparse} : effective sparse-kernel compute rate; BW_{sparse} : effective sparse-kernel bandwidth.
- ▶ B_x, B_w, B_y : input, weight, and output traffic; B_{idx} : index or metadata traffic needed to locate sparse entries.
- ▶ **Interpretation:** sparse models pay not only arithmetic cost, but also indexing, irregular memory access, and kernel overhead.
- ▶ If the runtime still executes dense kernels, shapes do not shrink and the model may be smaller on disk but not faster in latency.

Bottom line: Sparse FLOP reduction is not a latency claim unless the runtime and hardware exploit the **specific sparse pattern**.

How modern LLM pruning methods cluster

Cluster	Core mechanism	Representative methods
One-shot unstructured pruning	OBS-like local reconstruction or cheap saliency scores	SparseGPT, Wanda
Group / structural pruning	Remove heads, hidden dimensions, or block structure, often with short recovery tuning	LLM-Pruner, fluctuation-based adaptive structured pruning (FLAP)
Dimension reduction to smaller dense models	Rotate into removable coordinates and delete low-energy directions	SliceGPT
Hardware-supported semi-structured pruning	Enforce patterns like 2:4	Sparse tensor-core pipelines

- ▶ **Wanda** uses a very cheap score $|w_{ij}| \cdot \|x_j\|_2$, which is a diagonal-Hessian simplification of second-order pruning.
- ▶ **SliceGPT** is important because it yields a *smaller dense model*, which is much easier to accelerate faithfully than generic unstructured sparsity.

When pruning is the right move

Use pruning when one of the following is true:

- ▶ you can remove whole structures so dense kernels become cheaper;
- ▶ your hardware supports a regular sparse pattern such as $N:M$;
- ▶ you want compressed storage and are willing to accept weaker latency gains.

Avoid generic unstructured sparsity as a default deployment recipe when:

- ▶ the target runtime is optimized for dense kernels;
- ▶ your claim depends on theoretical FLOP reduction rather than measured end-to-end latency;
- ▶ a distilled smaller dense model or structured dimension reduction is allowed.

Bottom line: For mainstream deployment, prefer **structured pruning** or **dimension reduction**; use generic unstructured sparsity only with a hardware story.

Why distillation often deploys better than compression alone

Distillation trains a **new student model** to imitate a stronger **teacher**.

- ▶ Quantization and pruning modify an existing trained model.
- ▶ Distillation changes the training target so a **smaller dense architecture** can succeed.

Why dense students are attractive

A smaller dense student still uses mature dense kernels, often with cleaner latency, throughput, and memory behavior than a heavily compressed large model.

- ▶ Use distillation when the deployed architecture may change.
- ▶ Use LoRA-like methods later when the backbone must stay mostly fixed and only cheap adaptation is needed.

Bottom line: If retraining is allowed, distillation often gives the best **quality–latency frontier**.

Logits, softmax, and cross-entropy: the bridge to distillation

For class logits $z \in \mathbb{R}^K$, softmax produces probabilities

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}.$$

If the correct label is the one-hot vector y , the usual supervised cross-entropy (CE) loss is

$$\text{CE}(y, p) = - \sum_{i=1}^K y_i \log p_i.$$

- ▶ z_i is the model's unnormalized score for class i ; p_i is the resulting probability.
- ▶ If y is one-hot, cross-entropy is just the negative log-probability of the correct class.
- ▶ **Why this matters for distillation:** hard-label training uses only y , while distillation also asks the student to match a **soft teacher distribution**.
- ▶ A teacher can reveal that class 2 is much more plausible than class 3 even when both are wrong.

Bottom line: Distillation changes not only *which answer* is correct, but also *how probability mass is distributed* among alternatives.

The canonical knowledge distillation (KD) objective

Let z^T and z^S be teacher and student logits. Define

$$p^S = \text{softmax}(z^S), \quad p_\tau^T = \text{softmax}(z^T / \tau), \quad p_\tau^S = \text{softmax}(z^S / \tau).$$

Standard KD minimizes a cross-entropy term plus a Kullback–Leibler (KL) term:

$$\mathcal{L}_{\text{KD}} = (1 - \alpha) \text{CE}(y, p^S) + \alpha \tau^2 \text{KL}(p_\tau^T \parallel p_\tau^S).$$

- ▶ y is the hard label; α trades off hard-label supervision and teacher imitation.
- ▶ τ is the temperature; larger τ softens the teacher and student distributions.
- ▶ **Interpretation:** the student learns both which answer is correct and how the teacher distributes probability over plausible alternatives.

Why soft targets help

A teacher distribution can reveal that class 2 is more plausible than class 3 even when both are wrong, giving a richer supervision signal than a one-hot label.

Why the τ^2 factor appears

For the soft KD term, the student-logit gradient satisfies

$$\frac{\partial}{\partial z_i^S} \text{KL}(q \| p_\tau^S) = \frac{1}{\tau} (p_i - q_i), \quad q = p_\tau^T.$$

As τ increases, the softened probabilities become flatter, so $(p_i - q_i)$ also shrinks by roughly another factor of $1/\tau$.

- ▶ **Interpretation:** without compensation, the soft-target gradient becomes much smaller at high temperature.
- ▶ Multiplying the KL term by τ^2 keeps its scale comparable across temperatures.
- ▶ **Why it matters:** temperature is not only about smoother probabilities; it also changes optimization dynamics.

Bottom line: High temperature exposes teacher similarity structure, and the τ^2 factor keeps that signal numerically relevant during training.

Distillation is not only about final logits

Target	Typical loss	What the student learns
Final logits	KL or soft cross-entropy	Teacher output distribution
Hidden states	ℓ_2 or cosine regression	Intermediate representations (FitNets, TinyBERT-style)
Attention / relations	mean squared error (MSE) or KL on relation matrices	How tokens interact internally (MiniLM-style)
Rationales or intermediate text	Token-level cross-entropy	A decomposition of the reasoning process

- ▶ **FitNets:** match internal teacher features, not only final predictions.
- ▶ **MiniLM-style relation distillation:** teach the student token-to-token interaction geometry.

Bottom line: A small student may benefit more from matching the teacher's **structure of reasoning** than from matching the final probabilities alone.

Generative distillation: token-level KD is not enough

For an autoregressive model,

$$P_{\theta}(y_{1:T} \mid x) = \prod_{t=1}^T p_{\theta}(y_t \mid x, y_{<t}).$$

A forward-KL sequence objective is

$$\text{KL}(P_T \parallel P_S) = \mathbb{E}_{y \sim P_T} \left[\sum_{t=1}^T \log \frac{p_T(y_t \mid x, y_{<t})}{p_S(y_t \mid x, y_{<t})} \right].$$

- ▶ **Problem:** the expectation is over **teacher trajectories**. At inference, the student conditions on its **own** prefixes.
- ▶ This creates a train–inference mismatch known as **exposure bias**.
- ▶ Modern work also asks whether the divergence should remain forward KL; **reverse KL** is more mode-seeking and can be better aligned with low-capacity generative students.

Modern fix

Use **on-policy** distillation: compare teacher and student distributions on student-generated prefixes. This is the conceptual move behind **MiniLLM** and **GKD** (generalized knowledge distillation).

Modern generative KD families

Family	What changes?	Representative methods
Classical KD	Match teacher token distributions on dataset or teacher prefixes	Hinton KD, DistilBERT-style KD
Feature / relation KD	Match hidden states, attention relations, or intermediate text	FitNets, TinyBERT, MiniLM, rationale distillation
Sequence-level KD	Distill whole sequences or teacher-generated pseudo-targets	Sequence KD
On-policy generative KD	Distill on student-generated trajectories; sometimes change the divergence	MiniLLM, GKD

- ▶ **Organizing principle:** the first three rows change *what signal is distilled*; the last row changes *whose prefixes or trajectories are used* during training.
- ▶ Distinctions that matter: **what target is matched**, **which divergence is used**, and **whose trajectory distribution is used**.

Bottom line: For generative students, the extra question is not only *what* to match, but also *on whose prefixes* the match is enforced.

How to evaluate a distilled student for deployment

Judge a distilled student on three axes simultaneously:

- ① **Task quality:** downstream metrics, not only KD loss or perplexity.
 - ② **Architecture efficiency:** parameter count, activation size, and KV-cache footprint of the student itself.
 - ③ **Serving performance:** TTFT, TPOT, throughput, and memory on the same runtime stack used for the baseline.
- ▶ A common fallacy is to claim distillation “worked” because the KL term fell even though the student architecture still misses the latency budget.
 - ▶ For deployable students, the **architecture choice** and the **distillation loss** should be designed together.

Bottom line: Distillation succeeds only when the **student architecture** lands on the desired quality–latency operating point.

Low rank: intuition, compression, and adaptation

A rank-1 matrix has the form

$$W = uv^\top, \quad u = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \quad v^\top = \begin{bmatrix} 1 & 3 \end{bmatrix} \quad \Rightarrow \quad uv^\top = \begin{bmatrix} 2 & 6 \\ 1 & 3 \end{bmatrix}.$$

- ▶ Every column is a multiple of u and every row is a multiple of $v^\top$.
- ▶ **Interpretation:** the matrix has only one independent interaction direction.

For larger matrices, a rank- r factorization uses

$$W \approx UV^\top, \quad U \in \mathbb{R}^{m \times r}, \quad V \in \mathbb{R}^{n \times r},$$

so storage drops from mn parameters to about $r(m+n)$ when $r \ll \min(m, n)$.

- ▶ **Compression view:** approximate the deployed weight matrix itself by low rank.
- ▶ **Adaptation view:** keep a pretrained W_0 fixed and learn only a low-rank update.

Bottom line: Low rank helps when a few directions explain most of the matrix's action; compression and adaptation use the same algebra for different deployment goals.

Low-rank compression via the singular value decomposition

The classical rank-constrained problem is

$$\arg \min_{\text{rank}(\tilde{W}) \leq r} \|W - \tilde{W}\|_F^2.$$

If

$$W = U\Sigma V^\top, \quad \Sigma = \text{diag}(\sigma_1, \sigma_2, \dots), \quad \sigma_1 \geq \sigma_2 \geq \dots,$$

then the best rank- r approximation is

$$\tilde{W} = U_r \Sigma_r V_r^\top, \quad \|W - \tilde{W}\|_F^2 = \sum_{i>r} \sigma_i^2.$$

- ▶ This is the **Eckart–Young theorem**.
- ▶ **Interpretation:** keep the strongest singular directions and discard the weaker ones.
- ▶ **Why it matters:** when the singular spectrum decays quickly, much of the matrix can be represented with fewer effective degrees of freedom.

LoRA: low-rank adaptation with a mergeable inference path

LoRA (low-rank adaptation) assumes the fine-tuning update has low intrinsic rank:

$$\min_{\Delta W} \mathcal{L}(W_0 + \Delta W) \quad \text{s.t.} \quad \text{rank}(\Delta W) \leq r.$$

Instead of solving this directly, LoRA parameterizes

$$\Delta W = BA, \quad B \in \mathbb{R}^{m \times r}, \quad A \in \mathbb{R}^{r \times n}, \quad W = W_0 + \frac{\alpha}{r} BA.$$

- ▶ W_0 is the frozen pretrained weight; r is a small rank budget.
- ▶ **Interpretation:** train only a thin update rather than the full dense matrix.
- ▶ After training, the adapter can often be merged:

$$W_{\text{merged}} = W_0 + \frac{\alpha}{r} BA.$$

Bottom line: LoRA is popular because it is cheap to train and often adds essentially no extra inference cost after merging.

Quantized LoRA (QLoRA) and LoRA-Fine-Tuning-Aware Quantization (LoftQ): adapt a quantized backbone

- ▶ **QLoRA**: store the base model in 4-bit **NF4** (NormalFloat 4) or a similar low-bit format, dequantize for compute, and train only LoRA adapters.
- ▶ **Why it matters**: adaptation memory drops dramatically while the deployed architecture stays close to the base model.

A compact abstraction of **LoftQ** is

$$\min_{\hat{W} \in \mathcal{Q}, A, B} \left\| W - \hat{W} - AB \right\|_F^2.$$

- ▶ $\hat{W}$ is the quantized backbone; AB is a low-rank correction.
- ▶ **Interpretation**: jointly choose the quantized base and the adapter so they fit together from the start.
- ▶ **Why it matters**: a naive quantized backbone may be a poor starting point for low-rank adaptation; LoftQ reduces that mismatch.

Deployment reading

QLoRA is mostly a **training-memory optimization**; it matters operationally because it makes cheap adaptation of deployable large models feasible on modest hardware.

Beyond vanilla LoRA: what later variants change

Variant direction	What changes?	Why it can help
Adaptive rank allocation	Reallocate a fixed total rank budget across layers during training	More capacity where the task actually needs it (AdaLoRA)
Weight decomposition	Separate magnitude and direction instead of modeling everything with one low-rank update	Better optimization when vanilla LoRA is too rigid (DoRA)

- ▶ QLoRA and LoftQ answer a different question: how to adapt a **quantized backbone** efficiently; that was the previous frame.
- ▶ Most later **parameter-efficient fine-tuning (PEFT)** variants change either **where rank budget goes** or **how the update is parameterized**.

Bottom line: Once LoRA is understood, most follow-on variants are targeted refinements rather than new method families.

The right metrics for modern inference

For a set of completed requests $\mathcal{R}$ over wall time T_{wall} ,

$$\text{throughput}_{\text{req/s}} = \frac{|\mathcal{R}|}{T_{\text{wall}}}, \quad \text{throughput}_{\text{tok/s}} = \frac{\sum_{r \in \mathcal{R}} L_r^{\text{out}}}{T_{\text{wall}}}.$$

A useful online metric is **goodput**:

$$\text{goodput} = \frac{|\{r \in \mathcal{R} : \text{TTFT}_r \leq \tau_1, \text{TPOT}_r \leq \tau_2\}|}{T_{\text{wall}}}.$$

- ▶ L_r^{out} is output length; τ_1, τ_2 are latency service-level objectives.
- ▶ Minimum useful report: end-to-end latency, **TTFT** (time to first token), **TPOT** (time per output token) / **ITL** (inter-token latency), throughput, goodput, memory footprint, and ideally energy.
- ▶ Report p50/p95/p99 alongside means, especially for interactive services.

Bottom line: A system can have high raw throughput but low **goodput** if it misses latency targets.

Operating points: roofline for kernels, Little's law for services

Kernel view:

$$T_{\text{layer}} \gtrsim \max\left(\frac{F}{P_{\text{eff}}}, \frac{B}{BW_{\text{eff}}}\right).$$

Service view:

$$L = \lambda W.$$

- ▶ F : arithmetic operations; B : bytes moved; P_{eff} : effective compute rate; BW_{eff} : effective memory bandwidth.
- ▶ L : average concurrency in the system; λ : throughput; W : average latency.
- ▶ **Interpretation:** the first equation diagnoses a kernel bottleneck; the second says throughput and latency must be read at a stated concurrency point.

Why this is crucial

A benchmark can make a system look “fast” by choosing a throughput-favoring concurrency point, even though per-request latency is much worse. Conversely, a single-stream latency number says little about datacenter throughput.

Bottom line: Always ask **at what operating point?** before comparing any latency or throughput claim.

FlashAttention: exact attention with less memory traffic

Naive attention computes

$$S = \frac{QK^T}{\sqrt{d}}, \quad P = \text{softmax}(S), \quad O = PV,$$

where $Q, K, V \in \mathbb{R}^{n \times d}$, $S \in \mathbb{R}^{n \times n}$ is the score matrix, $P \in \mathbb{R}^{n \times n}$ the attention-probability matrix, and O the output.

- ▶ **Naive bottleneck:** materializing S and often P – that is, explicitly storing those full intermediate matrices in **high-bandwidth memory (HBM)** – creates heavy memory traffic.
- ▶ **FlashAttention idea:** process Q, K, V in tiles and keep only a small working set in fast on-chip memory.

For one query row, the kernel keeps a running max and normalizer:

$$m^{(t+1)} = \max(m^{(t)}, \max_j S_j^{(t+1)}), \quad \ell^{(t+1)} = e^{m^{(t)} - m^{(t+1)}} \ell^{(t)} + \sum_j e^{S_j^{(t+1)} - m^{(t+1)}}.$$

- ▶ $S_j^{(t+1)}$ is the score for key j inside tile $t + 1$.
- ▶ $m^{(t)}$ is the running row maximum; $\ell^{(t)}$ is the running softmax denominator after tile t .
- ▶ **Interpretation:** exact softmax can be accumulated tile by tile without storing the full $n \times n$ matrix in HBM.
- ▶ **Why it matters:** FlashAttention changes the kernel, not the model function, yet can yield large speedups for long-context inference. FlashAttention-2 keeps the same exact function but improves work partitioning and parallelism further.

Bottom line: Input/output (IO)-aware kernels can beat model-side approximations when memory traffic is the true bottleneck.

Speculative decoding: draft, verify, correct

- 1 A cheap **draft model** q proposes a block of future tokens.
- 2 The expensive **target model** p scores that drafted prefix in parallel.
- 3 Drafted token $\tilde{y}_i$ is accepted with probability

$$a_i = \min\left(1, \frac{p_i(\tilde{y}_i)}{q_i(\tilde{y}_i)}\right).$$

- 4 If rejection occurs, sample a correction token from

$$r_i(v) \propto [p_i(v) - q_i(v)]_+.$$

- $p_i(\cdot)$ and $q_i(\cdot)$ are the target and draft next-token distributions at step i .
- v ranges over vocabulary items, and $[u]_+ = \max(u, 0)$.
- **Interpretation:** the draft is cheap; the gain comes from reducing expensive target-model decode passes. If draft and target agree, one target pass can finalize several output tokens at once.
- **Exactness:** the acceptance-correction rule preserves the target model's output distribution.
- If the average accepted block length is $\bar{k}$, target passes per emitted token drop from about 1 to about $1/\bar{k}$.

Representative variants

External-draft speculation uses a separate small model; **Medusa**-style speculation adds lightweight proposal heads to the backbone itself.

PagedAttention and continuous batching

A contiguous KV-cache allocator can waste memory when different sequences use different context lengths. With fixed-size pages of P tokens, a sequence of length T_s needs

$$n_s = \left\lceil \frac{T_s}{P} \right\rceil \quad \Rightarrow \quad n_s P - T_s < P.$$

- ▶ **Interpretation:** internal fragmentation per sequence is bounded by one page, not by the gap to a worst-case reserved context length.
- ▶ Lower fragmentation lets more active sequences fit in memory.
- ▶ That, in turn, enables stronger **continuous batching** and higher throughput under realistic heterogeneous workloads.

Why this is a systems lesson

PagedAttention changes the runtime policy and memory manager, not the model weights, yet it can yield order-one throughput gains.

Bottom line: For long-context serving, KV-cache management can matter as much as model compression.

Benchmark correctly: protocol and common traps

A trustworthy benchmark should fix and report:

- ▶ hardware, driver/runtime, compiler, and serving framework;
- ▶ exact model version, precision, and the actual kernel path;
- ▶ prompt/output-length distributions and batch size or concurrency;
- ▶ warmup policy, synchronized timing, and p50/p95/p99;
- ▶ quality checks after optimization.

Common trap	Correct fix
Different workloads or decoding settings	Hold the workload fixed and state it explicitly
Sparse or low-bit claim without kernel confirmation	Verify that the intended kernel path really executes
Average latency only	Report tail latency and, for online serving, goodput
Offline throughput used as interactive-latency evidence	Report both at their own operating points
Quality judged by perplexity alone	Check downstream task quality too

- ▶ For LLMs, also separate **prefill** and **decode**; these can have different bottlenecks and different gains.

Bottom line: A benchmark is part of the method: changing the stack or workload changes the result.

A practical deployment playbook

- ❶ **Benchmark the dense baseline.** Identify whether the real bottleneck is compute, bandwidth, KV-cache capacity, or queueing/scheduling.
- ❷ **Fix runtime inefficiencies first.** Mature attention kernels and KV-cache managers can produce order-one gains before any model surgery.
- ❸ **Try quantization next.** It is usually the quickest route to lower memory and better bandwidth efficiency.
- ❹ **Use sparsity only with a hardware story.** Prefer structured pruning or smaller dense models when sparse kernels are weak.
- ❺ **Distill if architecture change is allowed.** Dense students often give the cleanest quality–latency trade-off.
- ❻ **Use LoRA/QLoRA for cheap task adaptation.** Merge adapters when appropriate.

Rule of thumb

If a claim stops at parameter count or FLOPs, it is not yet a deployment result.

Final synthesis

- ▶ Match the method to the bottleneck: quantization attacks bytes moved, structured pruning changes shapes, distillation changes the architecture, low-rank methods constrain weights or updates, and runtime kernels or schedulers change how work is executed.
- ▶ In practice, fix runtime bottlenecks first, then try quantization; use sparsity only with a hardware story; distill when architecture change is allowed; use LoRA/QLoRA for cheap adaptation.
- ▶ Trust only same-stack, end-to-end measurements on the target workload.

Final lesson

An efficiency claim is real only if it survives the actual deployment bottleneck, not just a proxy such as parameter count or theoretical FLOPs.

Appendix: notation and math toolkit cheat sheet

Idea / symbols	Formula or notation	Why it reappears
Layer output	$y = Wx + b$	Base operation changed by quantization, pruning, and low-rank structure
Reconstruction error	$\ WX - \hat{W}X\ _F^2$	Canonical PTQ and some one-shot pruning objectives
Local quadratic loss	$\mathcal{L}(\theta + \delta) \approx \mathcal{L}(\theta) + g^\top \delta + \frac{1}{2} \delta^\top H \delta$	Explains second-order saliency and mixed precision
KL divergence	$\text{KL}(P\ Q)$	Canonical way to compare teacher and student distributions
Low rank / singular value decomposition (SVD)	$W = U\Sigma V^\top$	Compression and LoRA-style adaptation
Little's law	$L = \lambda W$	Relates latency, throughput, and concurrency

- Repeated symbols: $\theta, \tilde{\theta}$ = original/transformed parameters; $W, \hat{W}$ = full-precision/modified weights; X = calibration activations; H = Hessian or surrogate; m = pruning mask; r = rank budget.

Appendix: two useful PTQ refinements

AdaRound. Instead of hard round-to-nearest, optimize the rounding choice itself:

$$\hat{w}(v) = s(\lfloor w/s \rfloor + h(v)), \quad \min_v \left\| WX - \hat{W}(v)X \right\|_F^2 + \lambda R(v).$$

- **Interpretation:** rounding is an optimization variable, not a fixed preprocessing step.

BRECQ (Block REConstruction for Quantization). Reconstruct whole residual blocks rather than isolated scalars:

$$\min_{\hat{W}_B} \left\| Y_B - f_B(X_B; \hat{W}_B) \right\|_F^2.$$

- **Interpretation:** the natural functional unit for quantization is often a block, not a single layer row or scalar.

Appendix: PACT and learnable clipping

PACT makes the activation clipping threshold trainable. For a nonnegative activation a and threshold $\alpha > 0$,

$$\text{PACT}_{\alpha}(a) = \frac{1}{2}(|a| - |a - \alpha| + \alpha) = \min(\max(a, 0), \alpha).$$

After clipping, the activation is quantized in the usual way.

- ▶ α is learned jointly with the network instead of chosen by hand.
- ▶ **Interpretation:** the model learns how much activation range it actually needs before low-bit rounding.
- ▶ **Why it matters historically:** PACT is a clean bridge from hand-designed clipping rules to modern QAT with learnable quantizer parameters.

Appendix: why mixed precision uses curvature

For layer ℓ with perturbation covariance Σ_ℓ ,

$$\mathbb{E}[\Delta\mathcal{L}_\ell] \approx \frac{1}{2} \text{tr}(H_\ell \Sigma_\ell).$$

If quantization noise is roughly isotropic, $\Sigma_\ell \approx \sigma_\ell^2 I$, then

$$\mathbb{E}[\Delta\mathcal{L}_\ell] \approx \frac{1}{2} \sigma_\ell^2 \text{tr}(H_\ell).$$

- **Interpretation:** layers with larger Hessian trace are more sensitive and deserve more bits.
- This is the key historical idea behind **HAWQ** (Hessian AWare Quantization)-style mixed-precision allocation.

Appendix: network slimming in one equation

With batch-normalized channels, let γ_c be the scale of channel c . Network Slimming optimizes

$$\min_{\theta, \gamma} \mathcal{L}(\theta, \gamma) + \lambda \sum_c |\gamma_c|.$$

The proximal update is scalar soft-thresholding:

$$\gamma_c^+ = \text{sign}(v_c) \max(|v_c| - \eta\lambda, 0).$$

- **Interpretation:** training itself pushes unimportant channel scales exactly to zero.
- After training, near-zero channels can be removed entirely, giving a structured hardware-friendly model.

Appendix: forward KL vs. reverse KL for generative KD

Forward KL on teacher trajectories:

$$\text{KL}(P_T \| P_S) = \mathbb{E}_{y \sim P_T} \left[\log \frac{P_T(y | x)}{P_S(y | x)} \right].$$

Reverse KL on student trajectories:

$$\text{KL}(P_S \| P_T) = \mathbb{E}_{y \sim P_S} \left[\log \frac{P_S(y | x)}{P_T(y | x)} \right].$$

- ▶ Forward KL is more **mode covering**: it pushes the student to cover the teacher distribution.
- ▶ Reverse KL is more **mode seeking**: it emphasizes where the student already places mass.
- ▶ Modern methods such as **MiniLLM** and **GKD** differ partly in which divergence and which trajectory distribution they optimize.

Appendix: feature, relation, and rationale distillation

A compact template for richer distillation is

$$\mathcal{L} = \mathcal{L}_{\text{KD}} + \lambda_h \|Ph_S - h_T\|_2^2 + \lambda_R \|R_S - R_T\|_F^2 + \lambda_r \text{CE}(r, \hat{r}).$$

- ▶ $\mathcal{L}_{\text{KD}}$ is the usual logit-level distillation term.
- ▶ h_T, h_S are teacher and student hidden states; P aligns dimensions when the student is narrower.
- ▶ R_T, R_S are relation matrices, often attention logits or token-token similarity maps.
- ▶ r is a teacher-generated rationale or intermediate text.
- ▶ **FitNets**: emphasize the hidden-state term.
- ▶ **MiniLM-style relation distillation**: emphasize the relation term.
- ▶ **Distilling Step-by-Step**: add rationale supervision during training, then deploy only the student.

Bottom line: Once the student is smaller, matching only final logits may be too weak; richer intermediate targets can recover part of the teacher's internal structure.

Appendix: representative lineage by family

Family	Historical roots	Representative modern lines
Quantization	Integer-only inference, PACT, LSQ, HAWQ, AdaRound, BRECQ	GPTQ, SmoothQuant, AWQ, OmniQuant, LLM-QAT, QuaRot, SpinQuant
Pruning	OBD, OBS, magnitude pruning, Deep Compression, Network Slimming	SparseGPT, Wanda, LLM-Pruner, FLAP, SliceGPT
Distillation	Teacher-student compression, Hinton KD, FitNets, sequence KD	MiniLM, MiniLLM, GKD, rationale distillation
Low rank / PEFT	Early low-rank compression, SVD-based speedups	LoRA, QLoRA, LoftQ, AdaLoRA, DoRA
Runtime systems	Roofline and queueing principles	FlashAttention, speculative decoding, Medusa, vLLM/PagedAttention, MLPerf-style benchmarking